

CiT Backend Engineering Track

Week 13: API Security — JWT & Spring Security

Authentication, Authorisation, JSON Web Tokens & Securing Endpoints

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 2 — Specialisation (Weeks 9–20)
Week	Week 13
Instructor	Jim & Anderson
Duration	6 months 24 weeks
Tech Stack	Java 25, Spring Boot 4, Spring Security, PostgreSQL, Postman

Confidential — For CiT Trainees Only

Opening Prayer

We begin, as always, by acknowledging that every gift of understanding comes from God. Please bow your heads as we invite His presence into this session.

Prayer of Dedication

Heavenly Father,

Today we learn to guard what is entrusted to us — to watch the gates, to know who may come in and who may go out, and to protect the people whose information passes through our hands. Make us faithful gatekeepers, careful and just, remembering that behind every account is a real person who trusts us with something precious.

As Psalm 121:8 promises, "The Lord shall preserve your going out and your coming in from this time forth, and even forevermore." As You keep watch over us, teach us to keep faithful watch over the work You have given us.

May we be diligent students who do not just copy but truly understand. May we encourage one another when we struggle. And may everything we build be offered as service to others.

In Jesus' name we pray,

Amen.

Scripture for the Week

"The Lord shall preserve your going out and your coming in from this time forth, and even forevermore."

— Psalm 121:8 (NKJV)

Table of Contents

1. Welcome to Week 13
2. Authentication vs Authorisation
3. Stateless APIs — Why Tokens, Not Sessions
4. JSON Web Tokens (JWT)
5. Introducing Spring Security
6. Securing Endpoints — The Filter Chain
7. Issuing & Validating a JWT
8. Authorisation — Roles & Method Security
9. Key Takeaways, Homework & Exercises

1. Welcome to Week 13

You have completed the Spring Boot Core module — you can build a clean, well-structured, well-behaved API. But there has been a glaring gap: until now, anyone who knows the URL can call any endpoint. That is fine for learning; it is unacceptable for a system holding real people's data.

This week opens the API Design & Security module. Security answers two questions about every request: who are you (authentication), and what are you allowed to do (authorisation)? You will answer both using Spring Security and JSON Web Tokens (JWT) — the standard, modern approach for securing REST APIs.

This is some of the most important material in the whole track. A single security mistake can expose thousands of people's money or grades. Learn it carefully, and hold it with the seriousness it deserves.

What to Expect in Week 13

By the end of this session, you will be able to:

- Clearly distinguish authentication from authorisation
- Explain why REST APIs are stateless and use tokens instead of sessions
- Describe what a JWT is, its three parts, and why it is signed but not secret
- Explain how the Spring Security filter chain protects requests
- Configure which endpoints are public and which require authentication
- Issue a JWT at login and validate it on every request
- Restrict actions by role with endpoint rules and `@PreAuthorize`

2. Authentication vs Authorisation

These two words sound alike and are constantly confused, but they answer completely different questions. Getting them straight is the foundation of everything else.

2.1 Two Different Questions

Authentication	Authorisation
Who are you?	What are you allowed to do?
Proving your identity	Checking your permissions
Happens first, once (login)	Happens on each protected action
Example: logging in with a password	Example: only an admin may delete a student

2.2 The Airport Analogy

Picture travelling through an airport. At the first checkpoint you show your passport — that proves who you are. That is authentication. Later, your boarding pass decides which flight you may board and which seat is yours — that is authorisation. The passport says you are Alice; the boarding pass says Alice may sit in seat 14C on this flight, but not walk into the cockpit.

An API works the same way. First it confirms who is calling; then, for each action, it checks whether that caller is permitted to perform it.

Say Them Precisely

Engineers shorten these to "authn" (authentication — who) and "authz" (authorisation — what). Using the words precisely prevents real bugs: many security holes come from an app that authenticates a user correctly but then forgets to authorise the specific action they are attempting.

3. Stateless APIs — Why Tokens, Not Sessions

Before JWTs make sense, you need to know what they replaced and why. The answer is about how a server remembers who you are between requests.

3.1 The Old Way — Sessions

Traditional web apps use sessions. When you log in, the server creates a session record in its memory and hands your browser a session id in a cookie. On each later request the browser sends that cookie, and the server looks up "who is this?" in its memory. The server remembers you.

This works, but it has costs: the server must store every logged-in user's session, and if you run several servers (as real systems do), they must all share those sessions somehow. It is also awkward for mobile apps, which do not handle cookies the way browsers do.

3.2 The API Way — Stateless & Tokens

REST APIs are designed to be stateless: the server keeps no memory of the caller between requests. Instead, each request carries everything needed to identify who is making it — a token.

At login the server gives the client a token. From then on, the client sends that token with every request. The server verifies the token and knows who is calling — without storing anything. No session memory, no shared session store.

Sessions (stateful)	Tokens (stateless)
Server stores each logged-in user	Server stores nothing about the caller
Hard to scale across many servers	Any server can verify any request
Relies on cookies	Works for web and mobile alike

Stateless Means Scalable

Because a stateless server remembers nothing, you can run ten copies of it behind a load balancer and any copy can handle any request — it just verifies the token. This is how real systems serve millions of users, and it is why modern APIs almost always choose tokens.

4. JSON Web Tokens (JWT)

A JWT (pronounced "jot") is the token most APIs use. It is a compact, digitally signed string that carries a few facts about the user, which the server can trust because it signed them itself.

4.1 The Three Parts

A JWT is three base64 pieces joined by dots: header.payload.signature. It looks like one long string:

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiIyMDI0MDAxLn0.4pcPyMD09oIPSyXnrXCjTwXyr4
```

Part	Contains
Header	The token type and which signing algorithm was used
Payload	The claims — facts about the user (id, roles, expiry time)
Signature	A cryptographic seal proving the token was issued by this server and not altered

4.2 The Payload — Claims

The middle part carries claims. Decoded, it is simply JSON:

```
{
  "sub": "2024001",      // subject — who this token is about
  "roles": ["ROLE_STUDENT"], // what they may do
  "exp": 1735689600      // expiry — when it stops being valid
}
```

4.3 Signed, Not Secret

This is the point everyone must understand. A JWT is signed, not encrypted. The signature — made with a secret key only the server holds — lets the server detect if even one character was changed. But the payload is only base64-encoded, which anyone can decode and read.

Never Put Secrets in a JWT

Because anyone holding the token can read its payload, **never** put a password, a national ID, or any secret inside it. The signature stops tampering, not reading. Put

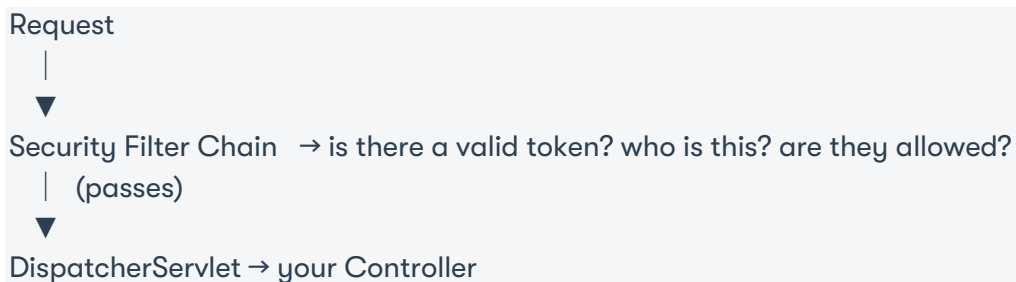
only what is safe to be seen — a user id, roles, an expiry — and keep the signing key itself strictly secret on the server.

5. Introducing Spring Security

Spring Security is the framework that handles authentication and authorisation in Spring applications. It is powerful and, at first, a little mysterious — so let us build the right mental model before touching configuration.

5.1 A Chain of Filters

Spring Security works as a chain of filters that sits in front of your controllers. Every incoming request passes through this security filter chain before it ever reaches your code. Each filter does one job — one checks for a token, another checks permissions — and only a request that passes them all reaches your controller.



5.2 Secure by Default

The moment you add Spring Security to a project, it locks everything down — every endpoint now requires authentication, and it even generates a login form. That is deliberate: security starts closed, and you deliberately open only what should be public. Our job is to configure it for a stateless JWT API rather than the default form login.

Do Not Fight the Framework

Spring Security has a reputation for being hard, mostly because beginners try to switch its protections off until things "work". Resist that. Its defaults are secure on purpose. Learn to configure it deliberately — opening exactly what must be open — rather than disabling it and exposing everything.

6. Securing Endpoints — The Filter Chain

Now we configure Spring Security for a stateless JWT API: declare which endpoints are open, which require a logged-in user, and switch off the session-based defaults that do not suit an API.

6.1 The SecurityFilterChain Bean

You define a SecurityFilterChain bean (recall beans from Week 9) that describes the rules.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(s -> s
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll()
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter,
                UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

6.2 Reading the Rules

Line	What It Says
csrf().disable()	Turn off CSRF protection — it targets cookie-based browser forms, not stateless token APIs
STATELESS	Never create an HTTP session — every request must carry its own token
requestMatchers("/auth/**").permitAll()	The login and register endpoints are public — no token needed to log in

<code>anyRequest().authenticated()</code>	Every other endpoint requires a valid token
<code>addFilterBefore(jwtFilter, ...)</code>	Run our JWT filter (Section 7) early to identify the caller

Only Disable CSRF Because We Are Stateless

Disabling CSRF is safe here precisely because the API is token-based and stateless — it does not rely on cookies a browser sends automatically. Never copy `csrf().disable()` into a traditional cookie-session app; there it removes a real protection.

7. Issuing & Validating a JWT

With the rules in place, two flows remain: giving a token at login, and checking it on every later request.

7.1 The Login Flow

The client posts credentials to the public login endpoint. If they are correct, the server generates a signed JWT and returns it. The client stores it and attaches it to every future request.

```
@PostMapping("/auth/login")
public ApiResponse<TokenResponse> login(@RequestBody LoginRequest req) {
    User user = userService.verify(req.username(), req.password());
    String token = jwtService.generateToken(user);
    return new ApiResponse<>("SUCCESS", "Login successful",
        new TokenResponse(token));
}
```

From then on, the client sends the token in the Authorization header on every request:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiYMDIOMDAXln0...
```

7.2 Validating on Every Request

The JWT filter you registered in Section 6 runs on each request. It reads the Authorization header, verifies the signature and expiry with the server's secret key, and — if valid — tells Spring Security who the caller is for the rest of that request.

1. Read the token from the Authorization header (strip the "Bearer " prefix)
2. Verify the signature with the secret key, and check it has not expired
3. If valid, set the authenticated user in Spring Security's context
4. If missing or invalid, the request is rejected with 401 Unauthorized

Passwords Are Never Stored in Plain Text

When you verify a login, you never compare raw passwords. Store only a one-way hash (Spring Security's BCryptPasswordEncoder is standard) and compare hashes. If your database ever leaks, hashed passwords cannot be read back into the originals. Storing plain-text passwords is a serious, unacceptable error.

8. Authorisation — Roles & Method Security

Authentication tells you who is calling. Authorisation decides what they may do — and it usually hinges on roles.

8.1 Roles

A role is a named permission group carried in the JWT, such as `ROLE_ADMIN` or `ROLE_STUDENT`. A student may read their own record; an admin may create and delete. You enforce roles in two places.

8.2 Restricting by Role in the Filter Chain

You can gate whole URL patterns by role in the security configuration:

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/auth/**").permitAll()
    .requestMatchers(HttpMethod.DELETE, "**").hasRole("ADMIN")
    .anyRequest().authenticated())
```

8.3 Method-Level Security with `@PreAuthorize`

For finer control, annotate individual methods. `@PreAuthorize` runs its check before the method executes.

```
@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) { ... }
```

A caller whose token lacks the `ADMIN` role gets `403 Forbidden` — authenticated (we know who they are) but not authorised (not allowed to do this). Note the difference from `401 Unauthorized`, which means "we do not know who you are at all".

Exercise: Secure the Library API With Two Roles

Apply this week to your Library Management System from Week 12:

- Add a public `/auth/login` endpoint that issues a JWT
- Require a valid token for every other endpoint
- Give `MEMBER` role read access, and `LIBRARIAN` role the right to add and delete books

- Confirm in Postman: no token → 401; wrong role → 403; correct role → success

9. Key Takeaways, Homework & Exercises

9.1 What You Learned This Week

- Authentication (who are you?) versus authorisation (what may you do?)
- Why REST APIs are stateless and use tokens instead of server-side sessions
- What a JWT is — its three parts, its claims, and that it is signed but not secret
- Spring Security as a filter chain that is secure by default
- Configuring a SecurityFilterChain for a stateless JWT API
- Issuing a JWT at login and validating it on every request (and hashing passwords)
- Authorising by role in the filter chain and with @PreAuthorize (401 vs 403)

9.2 Homework

5. Complete the in-class exercise and push it to GitHub
6. Secure your Library Management System: a login endpoint issuing JWTs, all other endpoints authenticated, and two roles with different permissions
7. Confirm with Postman that a missing token returns 401 and a wrong-role request returns 403, and never store a plain-text password
8. Preview Week 14: Read about the OWASP Top 10 and what SQL injection is. Come with one question ready.

Coming Next — Week 14: Secure Coding & Common Attacks

Jim & Anderson will continue the security module with secure coding practices: the most common attacks (SQL injection, cross-site scripting, and more from the OWASP Top 10), how to defend against them, and how to manage secrets safely. Authentication opened the door; now you learn to keep the whole house safe.

Closing Reflection

"Keep your heart with all diligence, for out of it spring the issues of life."

— Proverbs 4:23 (NKJV)

Security is, at its core, guarding what matters. You now hold the keys to people's data — their money, their records, their trust. Guard it with all diligence, as you would guard your own heart, and never treat lightly what has been entrusted to your care.

Glossary of Terms

Term	Definition
Authentication	Proving who a caller is — verifying identity, usually at login
Authorisation	Deciding what an authenticated caller is allowed to do
Bearer Token	A token sent in the Authorization header to identify the caller on each request
BCrypt	A one-way password hashing algorithm; store the hash, never the password
Claim	A fact carried in a JWT payload (subject, roles, expiry)
CSRF	Cross-Site Request Forgery — a cookie-based attack; not a threat to stateless token APIs
Filter Chain	The ordered set of Spring Security filters every request passes through first
JWT	JSON Web Token — a compact, signed token of header.payload.signature
Principal	The authenticated user Spring Security holds for the current request
Role	A named permission group (e.g. ROLE_ADMIN) used for authorisation
Signature	The cryptographic seal proving a JWT was issued by the server and not altered
Stateless	The server keeps no memory of a caller between requests; each request carries its own token
401 vs 403	401 Unauthorized = not authenticated; 403 Forbidden = authenticated but not permitted
@PreAuthorize	Restricts a method to callers meeting a security expression, e.g. a role